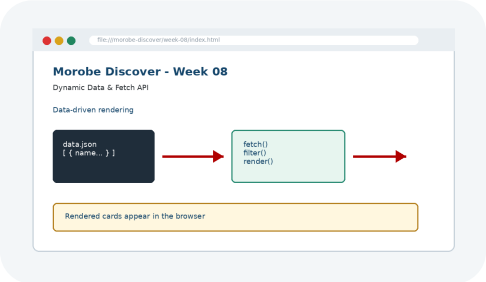
8 Dynamic Data & Fetch API

Prepared by Mr. Maxie Cletus Tepaiyan
School of Business Studies | PNG University of Technology



Objectives

You will have mastered the material in this chapter when you can:

- Decouple content from structure by loading and rendering dynamic external data.
- Identify how this week builds from the previous project checkpoint.
- Read and interpret key code snippets related to the feature.
- Explain how this week prepares the next project stage.
- Explain the key concepts and terminology for this week.
- Apply the new technique to the Morobe Discover project.
- Test the weekly project increment and record evidence.

Introduction

Dynamic Data & Fetch API introduces the next major layer of the Morobe Discover website. This chapter is written for students who are building the same project from start to final release. The topic is not treated as a separate exercise; it is a practical step in developing one working website.

Hard-coded cards become difficult to maintain when content grows or changes frequently. Professional web development requires students to understand why a technique exists, what problem it solves, and how it changes the behaviour or quality of a webpage.

Project - Morobe Discover

Morobe Discover is a tourism and local discovery website. Each week adds a working layer to the project. This week the project moves from **Week 07 added validation and saved lightweight user choices.** to **Destination and event cards are generated from JSON data instead of repeated static HTML.**

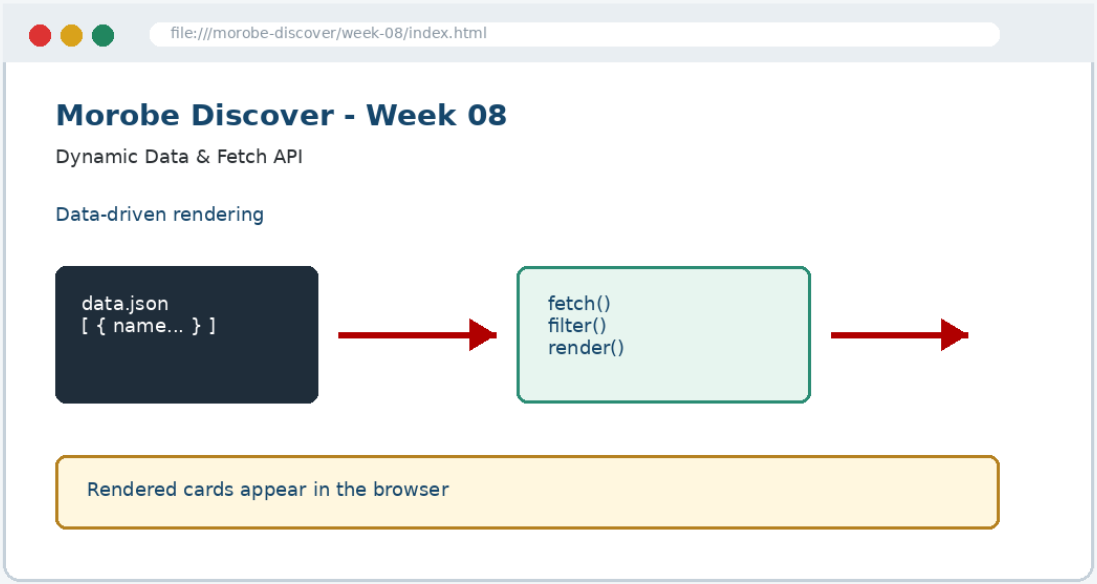


Figure 8-1. Current project focus for Week 08: Dynamic Data & Fetch API.

By the end of the chapter, the project evidence should clearly show the development focus for this week: **JSON content, Fetch API requests and data filters**.

Before This Week and After This Week

Before Week 08	After Week 08
Week 07 added validation and saved lightweight user choices.	Destination and event cards are generated from JSON data instead of repeated static HTML.
Project evidence from the previous checkpoint must be preserved.	The new feature becomes the starting point for Week 09.

Weekly Development Roadmap

1. Review current project
2. Understand the main concept
3. Inspect the interface problem
4. Study the key code
5. Apply it to the project
6. Test the result
7. Commit evidence
8. Prepare for next week

This roadmap mirrors a professional workflow. Students first inspect the current project state, then learn the concept, apply key code, test the visible result and commit the evidence.

1. Core Principle

Store repeatable content as data, fetch it, then render the interface from that data.

> **Developer Tip:** A good web developer can explain not only what code was written, but why that code is appropriate for the user task and project context.

Concepts Introduced This Week

Concept	Purpose	Project use
JSON represents structured content	Morobe Discover application	Used in Week 08 project evidence
fetch() loads external resources asynchronously	Morobe Discover application	Used in Week 08 project evidence
Filtering transforms data before rendering	Morobe Discover application	Used in Week 08 project evidence

The concepts in this chapter are introduced in small pieces. Each piece is immediately connected to the Morobe Discover project so that students can see how the idea becomes working project evidence.

2. Explain - Illustrate - Apply

The first step is to understand the interface or coding problem. A weak implementation usually focuses on surface appearance only. A stronger implementation identifies the structural or interaction principle and applies it consistently.

Weak approach

Add code until the page appears to work. Ignore structure, reusability or testing.

Professional approach

Use the correct concept, name the affected component, test the output and commit evidence.

Thinking Question

> **Question:** How does this week improve the website for the user, not just for the developer?
> **Answer:** It makes the current project easier to use, understand, interact with, test or release.

3. Key Code Pattern A

The following code shows the smallest useful pattern for this week's implementation. It is not a complete project file. The complete project belongs in the GitHub repository under ***/week-08/***.

```
const response = await fetch("data/destinations.json");
if (!response.ok) throw new Error("Data failed to load");
const destinations = await response.json();
```

Read the code line by line. Identify the selector, tag, function or command being used. Then identify the visible effect it should produce in the project.

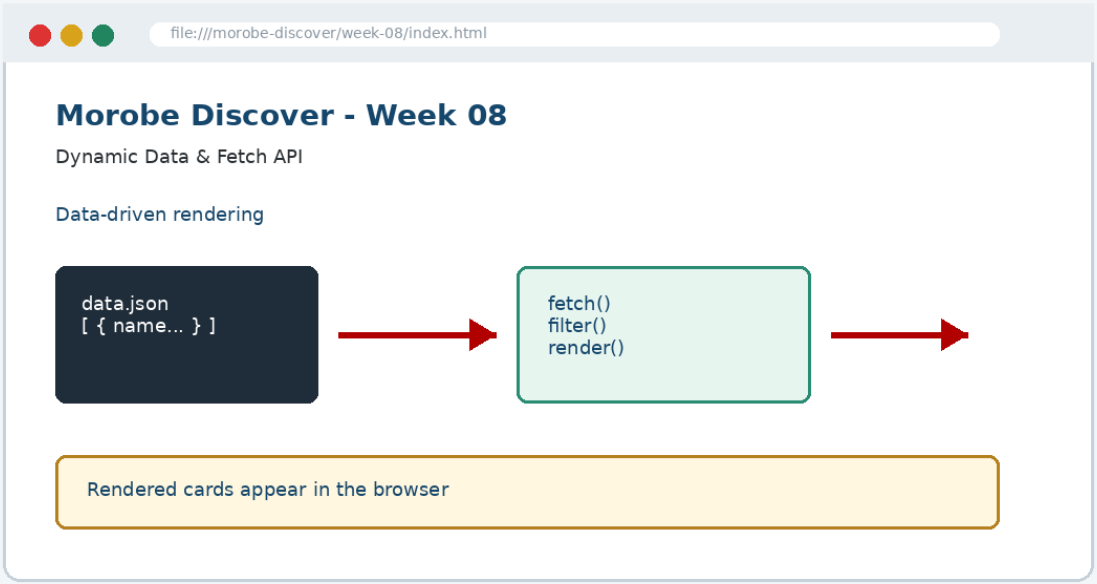


Figure 8-2. Code should be checked against visible output in the browser.

4. Key Code Pattern B

Most weeks require a second pattern that builds on the first. The second pattern usually applies the concept to a different component, state or project file.

```
const html = destinations
  .map(item => `<article class="card">${item.name}</article>`)
  .join("");
```

> **Good Practice:** Keep code readable, named and testable. Avoid copying code that you cannot explain during a code defence.

What to Inspect

- Which file should contain this code?
- Which part of the browser output should change?
- What error would appear if the code were placed in the wrong file?

5. Applying the Concept to Morobe Discover

Implementation should be completed as a project increment, not as an isolated demonstration. The following steps describe the development path.

To Build the Week 08 Feature

1. Create destinations.json
2. Fetch the JSON file from JavaScript
3. Render cards into the DOM
4. Add search or category filtering
5. Show loading and error states

Each step should be performed against the same Morobe Discover project folder. Do not create a separate mini-site for this week.

> **Project Rule:** Preserve earlier features. This week adds a layer to the project; it does not replace previous work.

6. Visual Result and Interface Evidence

The browser result is part of the evidence. Students should compare the interface before and after the week to confirm that the new concept has actually changed the site in the intended way.

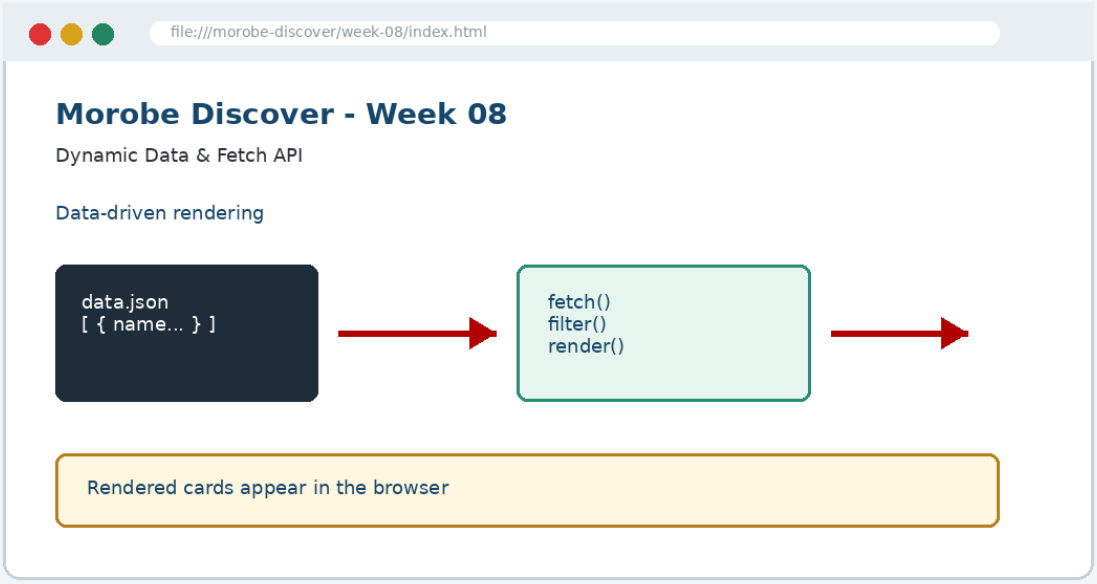


Figure 8-3. Expected Week 08 project output evidence.

Evidence item	What it proves
Screenshot	The feature is visible to the user.
Code file	The implementation exists in the correct location.
Test note	The student checked the behaviour.
Git commit	The project history records the checkpoint.

7. Common Mistake and Correction

> **Common Mistake:** Treating this week's topic as a definition to memorise instead of a technique to apply to the project.

Incorrect approach

The student can define the concept but cannot show where it appears in the Morobe Discover files or browser output.

Correction

Open the relevant project file, locate the code pattern, refresh the browser and explain the output. A student should be able to point to the exact line of code and the exact interface region affected.

```
// Explain during code defence
File: week-08/...
Feature: Dynamic Data & Fetch API
Evidence: browser screenshot + Git commit
```

8. Testing and Validation

Every weekly increment must be tested. Testing confirms that the feature is useful, not just present.

Check	Expected result
Run through a local web server	Expected project behaviour is visible and working.
Break the JSON path and observe the error state	Expected project behaviour is visible and working.
Test filters with empty results	Expected project behaviour is visible and working.
Check rendered cards preserve accessibility labels	Expected project behaviour is visible and working.

> **Incorrect result and correction:** If the page appears unchanged after editing a file, confirm that you edited the correct file, linked the file properly and refreshed the browser. Then inspect the browser console where relevant.

9. GitHub and Project Evidence

The complete implementation for this week should be stored in the project repository, not copied in full into this chapter. The chapter shows key code only so that students focus on understanding.

```
Suggested repository:
https://github.com/YOUR-USERNAME/is229-morobe-discover

Weekly folder:
/week-08/

Complete project:
/complete-project/
```

Commit Message Example

```
git add .
git commit -m "Week 08 Dynamic Data & Fetch API project increment"
```

10. Completed Weekly Outcome

At the end of Week 08, the project should demonstrate the following completed outcome:

> **Completed outcome:** JSON content, Fetch API requests and data filters

Definition of Done

Done when...	Evidence
The project includes the Week 08 feature.	Updated files in /week-08/
The feature can be explained using key concepts.	Student code explanation
The feature works in the browser.	Screenshot and test note
The work is version-controlled.	Git commit / repository history

11. Bridge to the Next Week

Week 09 checks usability, accessibility and the quality of the interactive site.

Week 08 output	Next use
JSON content, Fetch API requests and data filters	Week 09 checks usability, accessibility and the quality of the interactive site.

This bridge is important because the course uses one cumulative project. The work produced this week becomes part of the starting point for the next class.

Chapter Summary

This chapter introduced Dynamic Data & Fetch API as a practical development step in the Morobe Discover project. You learned the principle behind the topic, studied key code patterns, applied the concept to the project, tested the result and prepared evidence for GitHub.

Apply What You Learned

1. Explain what is gained by moving destination content from HTML into JSON.
2. Identify the exact project file that should change this week.
3. Explain how the browser output proves the feature works.
4. Write one test note for the weekly feature.
5. Explain how this week connects to the next week.

> **Before the next class:** Read the next week's material and bring your current project with this week's feature completed and committed.